SPARKLAB
(HarvestMind)

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

---

1. Introduction

1.1 Purpose

HarvestMind is an AI-driven decision intelligence platform that helps **Malaysian farmers** make **data-based** crop planning decisions by analysing **risk**, **cycle duration**, and **market trends**. Powered by the **ILMU GLM** (ilmu-glm-5.1) reasoning engine, it answers the core question every smallholder faces:|

"Should I persevere with my current crop, pivot to an alternative, or harvest early to protect revenue?"

This document covers the following key elements of the system:

Element	Summary
Architecture	Modular monolith with a Vanilla JS/Node.js frontend, FastAPI backend, PostgreSQL database, Celery/Redis worker layer, and a self-hosted Firecrawl scraper. Deployed via Docker Compose on Render.com.
Data Flows	Farmer input travels from the browser through a Node.js proxy to the FastAPI orchestrator, which calls Open-Meteo, GDELT, and FAMA concurrently. Normalised signals are stored in PostgreSQL and passed to the GLM reasoning layer.
Model Process	End-to-end workflow: conversational intake → field extraction → parallel data enrichment → deterministic analysis tools → GLM recommendation → persisted response streamed back via SSE.
Role of Reference	This document serves as the authoritative architectural reference for all developers, testers, data analysts, and hackathon judges evaluating HarvestMind.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Background

Malaysian smallholder farmers who make up over 80 % of the agricultural sector make crop and harvest decisions with limited access to real-time market intelligence, weather forecasts, or regional news signals. Most rely on informal networks or gut instinct, resulting in preventable losses from market oversupply, weather disruption, and poor timing.

HarvestMind was conceived to close this information gap by:

- Aggregating live market prices from **FAMA** (Federal Agricultural Marketing Authority)
- Fetching 7-day weather forecasts from Open-Meteo for precise Malaysian locations
- **Indexing regional agricultural news** from GDELT and Firecrawl
- Running all three signals through a structured GLM reasoning pipeline to produce a single, actionable recommendation

Previous / Baseline State:

Prior to HarvestMind there was no digital version of this system. Farmers used WhatsApp groups, manual market visits, and government SMS bulletins. There is no previous software version , this is a greenfield build. The architectural evolution from prototype to hackathon MVP is captured in the plan.md and ARCHITECTURE.md documents in the repository.

Major New Capabilities in this Version:

- Conversational guided intake (NL → structured fields via GLM extraction)
- Persisted intake sessions resumable by session ID
- **Real-time** decision **analysis** streamed via Server-Sent Events (SSE)
- **Live weather** snapshot **retrieval** (Open-Meteo)
- **Live news retrieval** and event-tagging (GDELT + Firecrawl fallback)
- FAMA web-scraping with Firecrawl fallback for market prices
- Full recommendation persistence with farmer **feedback capture**
- Celery/Redis background job scheduler for automated signal refresh

© UMHackathon 2026

1.3 Target Stakeholders

Stakeholder	Role	Expectations
Farmer / End User	Inputs crop data via conversational chat and receives AI recommendations on their dashboard	Simple mobile-friendly interface, clear Pivot/Persevere/Harvest Early decision, confidence score, and actionable next steps
Backend Developer	Builds and maintains the FastAPI orchestrator, repositories, GLM client, and Celery workers	Modular codebase with Repository pattern, documented APIs via /docs (Swagger), clear service boundaries
Frontend Developer	Builds and maintains the Vanilla JS pages and Node.js proxy server	Consistent design system, stable /api/v1 contract, SSE streaming support from backend
Data Analyst	Validates AI outputs, risk model logic, and evidence packets	Transparent data flow, persisted signal snapshots, recommendation history endpoint for audit
Judges / Reviewers	Evaluate technical feasibility, innovation, and impact at UMHackathon 2026	Clear architecture, measurable outcomes, demo-ready Docker Compose stack

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1 High-Level Architecture

2.1.1 Overview

Type	Details
System	Web Application (Desktop + Mobile Browser)
Architecture Pattern	Modular Monolith with a decoupled Worker Layer
Deployment	Docker Compose on Render.com (cloud-hosted)
Primary Language	Python 3.11 (backend) + JavaScript ES2022 (frontend)
AI Reasoning Engine	ILMU GLM 5.1 via api.ilmu.ai (OpenAI-compatible API)

HarvestMind is structured as a client-server system with four distinct layers:

- **Frontend** Layer , Five static HTML pages served by a **Node.js proxy (port 3000)** that also forwards **/api/v1/*** calls to the **FastAPI** backend and proxies GLM chat completions to protect the API key
- **Backend** API Layer , **Python FastAPI (port 8000)** organised into routes, services, repositories, clients, and deterministic tools. Exposes a RESTful API with one SSE streaming endpoint
- **Data** Layer , **PostgreSQL 16** for all persistent data (profiles, recommendations, sessions, signals, jobs); **Redis** for Celery broker and result backend
- **Intelligence** Layer , ILMU GLM 5.1 called from two places: the **GLMFieldExtractor** (intake) and the **GLMDecisionClient** (recommendation synthesis)

© UMHackathon 2026

2.1.2 LLM as Service Layer

GLM is integrated as a purposeful service layer , not as an autonomous agent. The system uses two targeted GLM calls per decision run:

GLM Call	Caller	Context Window Contents	Output
Field Extraction	GLMFieldExtractor (intake/field_extractor.py)	System prompt with field schema + asked_field + current intake state + farmer free-text answer. Max 1024 tokens output.	JSON object {updates: {...}} mapping extracted fields (location, crop, harvest_days, farm_size, labor_pct, candidate_crops)
Decision Synthesis	GLMDecisionClient (clients/glm.py)	System prompt (role + JSON contract) + user prompt containing: request summary (location, crop, harvest_days, farm_size, labor_pct) + evidence packet (price signal, weather signal, news signal, seasonal context, price snapshot, weather snapshot, top 5 news items). Temperature 0.1, response_format: json_object.	Structured JSON: {decision, price_outlook, revenue_risk, confidence, summary, reasons[3], actions[3], risks[3], next_review_days}

Token limits are enforced by:

- Capping news items to 5, price history to last 4 points, reasons/actions/risks to 3 items each
- Sending only the **fields relevant** to each call (no full conversation history)
- Setting max_tokens: 1024 for field extraction; relying on the json_object response format for the decision synthesis call
- The field extractor has a 20 s timeout; the decision client has a 120 s timeout with graceful degradation to a deterministic fallback recommendation if GLM fails

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.3 Dependency Diagram

```
BROWSER (port 3000)
 |
 ├── GET /index.html, /markets.html, /crops.html ──▶ Node.js Static Server
 |                                                   (server.js)
 ├── POST /api/chat  ──▶ Node.js Proxy  ──▶  api.ilmu.ai:443
 |                                                   (ILMU GLM 5.1)
 └── /api/v1/*  ──────▶ FastAPI (port 8000)
                              |
           ┌──────────────────┼──────────────┐
           |                  |              |
PostgreSQL 16            Redis 7        api.ilmu.ai
(farmers,                (Celery        (GLM Field
 sessions,               broker +       Extraction +
 recommendations,        result         Decision
 signals, jobs)          backend)       Synthesis)
                              |
           ┌──────────────────┼──────────────┐
           |                  |              |
Open-Meteo API           GDELT Doc API  FAMA Website
(weather,7-day)          (news, 14d)    (scraped prices)
                                |
Celery Worker + Scheduler  ◀────┘
(daily market, weekly signals,
 on-demand weather/news sync)


Self-hosted Firecrawl (port 3002)
Fallback scraper for prices + news
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
(firecrawl-api + playwright + rabbitmq)
```

Key API calls between components:

Source	Call	Target	Purpose
Browser	POST /api/chat	Node.js Proxy → api.ilmu.ai/v1/chat/completions	Free-chat GLM conversation with web_search tool
Browser	POST /api/v1/intake/sessions/:id/message	FastAPI intake router	Conversational farm data intake (persisted session)
Browser	POST /api/v1/analysis/decision (SSE)	FastAPI analysis router → Orchestrator	Trigger full decision run, stream progress + result
Browser	GET /api/v1/market/summary	FastAPI market router → SignalRepository	Fetch latest stored market prices
FastAPI Orchestrator	GET https://geocoding-api.open-meteo.com + forecast	Open-Meteo API	Resolve Malaysian location + fetch 7-day forecast

© UMHackathon 2026

Email: umhackathon@um.edu.my

FastAPI Orchestrator	GET https://api.gdeltproject.org/api/v2/doc/doc	GDELT Doc API	Fetch recent news articles for crop + region
FastAPI Orchestrator	GET fama.gov.my + Firecrawl /search	FAMA + Firecrawl	Fetch / scrape crop market prices
FastAPI Orchestrator	POST api.ilmu.ai/v1/chat/completions	ILMU GLM	Decision synthesis from assembled evidence
FastAPI GLMFieldExtractor	POST api.ilmu.ai/v1/chat/completions	ILMU GLM	Extract structured fields from farmer free-text
Celery Worker	Internal service calls	OperationsService / MarketService	Scheduled daily market refresh + weekly signal cards

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2 Sequence Diagram , Full Decision Analysis Flow

The following traces the primary user journey from farm intake through to receiving a recommendation:

```
Farmer      Browser        Node.js        FastAPI        GLM            Open-Meteo  GDELT
FAMA

  |             |          Proxy          API            API            API     API    Site

  |  Opens Chat |          |              |              |              |       |      |

  |————————————▶|          |              |              |              |       |      |

  |  POST /api/v1/intake/sessions (create session)

  |             |————————▶|————————▶|            |                  |       |      |

  |             |          |  201 session_id back to browser

  |             |◀————————|◀————————|            |                  |       |      |

  |  Asks 'Where is your farm?'

  |————————————▶|          |              |              |              |       |      |

  |  POST /intake/sessions/:id/message (message='Muar, Johor')

  |             |————————▶|————————▶|  POST chat  |                 |       |      |

  |             |          |              |————————▶| (extract       |          |   |

  |             |          |              |  location field) |         |       |

  |             |          |              |◀————————|                 |       |      |

  |  [Repeat for crop, harvest_days, farm_size, labor_pct, candidates]

  |  (6 intake questions answered)

  |  POST /api/v1/analysis/decision (SSE stream)

  |             |————————▶|————————▶|            |                  |       |      |

  |             | SSE: 'Gathering real-time intelligence...'

  |             |          | [asyncio.gather: 3 parallel calls]

  |             |          |              |————————————————▶| geocode+forecast
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
|            |         |              |──────────────────────►| news search
|            |         |              |──────────────────────────►| scrape prices
|            | SSE: 'Analysing weather and market prices...'
|            |         | [deterministic tools run]
|            |         | analyze_price_signal, analyze_weather_signal,
|            |         | analyze_seasonal_context, analyze_news_signal,
|            |         | build_scenario_comparison
|            | SSE: 'Synthesizing final AI recommendation...'
|            |         |              |  POST chat  |
|            |         |              |──────────►| (evidence packet)
|            |         |              |◄──────────| JSON decision
|            |         | [persist: signals, recommendation to PostgreSQL]
|  SSE: {result: {decision, confidence, reasons, actions...}}
|◄──────────|
|  Dashboard renders recommendation
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2 Technology Stack

Layer	Technology	Justification
Frontend	Vanilla JS / HTML5 / CSS3 + Node.js 20	Zero build-step static pages. Node.js acts as a secure proxy to protect API keys and forward /api/v1/* to the backend. Chosen for speed of development and minimal dependencies.
Backend API	Python 3.11 + FastAPI 0.115 + Uvicorn	FastAPI provides async request handling, automatic OpenAPI docs (/docs), and Pydantic validation on all inputs/outputs. Async is critical for parallel data fetching in the orchestrator.
Database	PostgreSQL 16 (psycopg3 binary)	Relational store for structured farmer profiles, recommendation runs, intake sessions, feedback, signal snapshots, and sync jobs. GIN indexes on JSONB evidence_packet columns for future retrieval.
Task Queue	Celery 5.4 + Redis 7	Celery Beat drives scheduled daily market price refresh and weekly GLM signal card generation. Redis serves as both the message broker and result backend.
AI / Intelligence	ILMU GLM 5.1 (ilmu.ai)	OpenAI-compatible API. Used for structured field extraction during intake and evidence-grounded decision synthesis. json_object response format ensures parseable output.
Web Scraping	Self-hosted Firecrawl + Playwright	Firecrawl scrapes FAMA market price pages and performs web searches when GDELT or direct HTTP fetches fail. Playwright handles JavaScript-rendered pages. Deployed as a sidecar service in Docker Compose.
Cloud / Deployment	Docker Compose + Render.com	Docker Compose orchestrates eight services: frontend, api, worker, scheduler, db (PostgreSQL), redis, firecrawl-api, firecrawl-playwright, firecrawl-rabbitmq, firecrawl-postgres. Render.com hosts the production stack.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)

The DFD below illustrates how data moves from external entities through the system and into persistent storage:

```
EXTERNAL ENTITIES          PROCESSES                      DATA STORES



Farmer                     [P1] Conversational            D1: intake_sessions

 |  (free text)            Intake Manager                 (state, transcript)

 └──────────────────►[P1]──►[P2] Field Extractor──► D2: farmers

                                  (GLM call)              (profile, preferences)



 [P3] Decision Orchestrator  ◄──────────────────► D2

  ├── [P4] Weather Fetch ──────► Open-Meteo API (external)

  ├── [P5] News Fetch ──────► GDELT API (external)

  └── [P6] Price Fetch ──────► FAMA / Firecrawl (external)

  |

  |  (evidence packet)

  ► [P7] GLM Decision Synthesis ─► api.ilmu.ai (external)

           | (structured JSON recommendation)

  ┌────────┼────────┐

  |        |        |

  |        |        |

  |        |        |

  |        |        |
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
  |         |               |

  |         |               |

  D3:       D4:             D5:

signal_      recommend-     sync_

snapshots    ation_runs     jobs



    ▼ (SSE stream , no black hole: all data persisted before streaming)

  Farmer ◄——— Browser ◄——— Node.js Proxy ◄——— FastAPI



  D6: recommendation_feedback  (Farmer outcome ratings, post-harvest)
```

Data integrity note:

All **data flows obey** the **DFD Black-Hole rule**. The analysis or decision SSE endpoint writes signal_snapshots and a recommendation_run to PostgreSQL before emitting the final result event, ensuring no processed data is lost even if the browser disconnects.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2 Normalised Database Schema (ERD Summary , 3NF)

The schema in api/sql/001_init.sql is designed to Third Normal Form (3NF). All repeating groups and transitive dependencies have been eliminated:

Table	Primary Key	Key Columns	Relationships / Notes
farmers	farmer_id (TEXT)	location, preferred_crops (JSONB), current_crop, current_price_rm_per_kg, expected_harvest_days, labor_flexibility_pct, candidate_crops (JSONB), latest_intake_state (JSONB)	Root entity. All other tables reference this via FK.
intake_sessions	session_id (TEXT)	farmer_id (FK), state (JSONB), analysis_request (JSONB), transcript (JSONB), status	M:1 with farmers. JSONB state stores the live intake progress; transcript stores the full conversation.
recommendation_runs	recommendation_id (UUID, gen_random_uuid())	farmer_id (FK), recorded_at, input_payload (JSONB), evidence_packet (JSONB), decision (CHECK constraint), confidence (NUMERIC 0-1)	M:1 with farmers. GIN indexes on input_payload and evidence_packet for future query/retrieval.
recommendation_feedback	feedback_id (UUID)	recommendation_id (FK nullable), farmer_id (FK), rating (1-5), outcome_label, actual_decision, actual_revenue_change_pct	Resolves M:N between farmers and recommendations. outcome_label is a controlled vocabulary.

© UMHackathon 2026

Email: umhackathon@um.edu.my

signal_snapshots	**snapshot_id (UUID)**	**farmer_id (FK nullable), signal_type (CHECK: price/weather/news), crop, location, source, normalized_payload (JSONB)**	**Stores normalised market, weather, and news signals. Can be farmer-scoped or global (farmer_id NULL for market-wide snapshots).**
sync_jobs	**job_id (UUID)**	**source (CHECK: market/weather/news), status (CHECK: queued/running/completed/failed), result_payload (JSONB), error**	**Audit log for Celery and on-demand sync operations.**

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

3.1 Minimum Viable Product

3.1.1 Core Features for Demo

#	Feature	Description	Endpoint / Component
1	Crop Risk Analysis	GLM evaluates persevere/pivot/harvest-early risk from farmer profile, live weather (Open-Meteo), news (GDELT), and market prices (FAMA). Produces a confidence-scored structured recommendation.	POST /api/v1/analysis/decision (SSE) → DecisionOrchestrator + GLMDecisionClient
2	Decision Dashboard (My Data)	Displays the latest Pivot/Persevere/Harvest Early recommendation for the demo farmer, including decision, confidence %, price signal, and data freshness indicators for price/weather/news signals.	GET /api/v1/recommendations/{farmer_id} + GET /api/v1/signals/{farmer_id}/latest → public/mydata.html
3	Data Visualisation (Markets)	Live price range table for 8 tracked Malaysian crops (Chili, Ginger, Kangkung, Tomato, Durian, Banana, Turmeric, Cucumber). Trend bars, 30-day % change, and Buy/Hold/Watch signal. GLM-generated weekly signal cards.	GET /api/v1/market/summary + /signals → public/markets.html
4	Conversational Intake Form	Guided 6-question intake flow. GLM extracts structured fields (location, crop, harvest_days, farm_size_ha, labor_flexibility_pct, candidate_crops) from free-text answers. Persisted server-side with session resume.	POST /api/v1/intake/sessions + /message → ConversationManager + GLMFieldExtractor

© UMHackathon 2026

Email: umhackathon@um.edu.my

5	Real-Time API Integration	Parallel async fetches of: (a) Open-Meteo 7-day weather forecast with geo-resolve for any Malaysian district; (b) GDELT news articles with relevance scoring; (c) FAMA price scraping with Firecrawl fallback. All results normalised and persisted as signal snapshots.	DecisionOrchestrator._fetch_weather/news/prices → Open-Meteo, GDELT, FAMA/Firecrawl

3.2 Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	Handle multiple concurrent user sessions without degradation	Docker Compose with Render.com horizontal scaling. Celery workers are stateless and can be replicated. PostgreSQL connection pooled via psycopg3. asyncio-based FastAPI handles concurrent /analysis/decision SSE streams.
Reliability	Maintain uptime during external API failures (Open-Meteo, GDELT, FAMA, GLM)	Composite clients with primary/fallback chains: FAMA → Firecrawl → seeded fallback for prices; GDELT → Firecrawl for news; deterministic fallback recommendation if GLM fails. Retry logic via httpx with explicit timeouts (15 s weather, 10 s news, 120 s GLM).
Maintainability	Modular, testable codebase separated by responsibility	Repository Pattern for all DB access. Services layer for business logic. Clients layer for external APIs. Tools layer for deterministic analytics. Routes are thin controllers. AGENTS.md codifies conventions.
Token Latency	GLM responses < 30 s; intake extraction < 20 s	Decision synthesis: temperature 0.1, json_object format, evidence packet truncated to critical fields only. Field extraction: max_tokens 1024, temperature 0, 20 s timeout. SSE streams progress messages to the user while the backend works, providing perceived responsiveness.

© UMHackathon 2026

Cost Efficiency	**Token spend per analysis run must remain bounded**	**Decision synthesis prompt is assembled deterministically (no open-ended context growth). Price history capped to last 4 observations. News items capped to 5. Reasons/actions/risks capped to 3 each. Weekly GLM signal cards (not per-request). Firecrawl searches are lazy , only triggered when primary data sources fail.**
Security	**API keys never exposed to the browser**	**Node.js proxy holds ILMU_API_KEY in process env and forwards it server-side only. Docker secrets via .env file excluded from git (.gitignore). CORS origins locked to localhost:3000 in development.**

3.3 Out of Scope / Future Enhancements

- Personalised **mobile application** (iOS / Android native)
- **User authentication** & **multi-tenant** account **management** (OAuth2/JWT)
- Advanced **long-term memory** / personalisation across sessions
- Real FAOSTAT or DOA price connector (currently using FAMA scraper + seeded fallback)
- **Exportable PDF** decision reports for banks and insurance providers
- **ROI** / cost-of-pivot **revenue calculations**
- pgvector hybrid retrieval over stored news and advisory documents
- App theme **customisation**

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

4.1.1 Grayscale Rollout & A/B Testing

For the hackathon MVP, all users access the same version of HarvestMind. The intended post-MVP release strategy is:

- New recommendation model versions or prompt changes are deployed to **5%** of farmer profiles (canary). Error rate and confidence score distributions are **monitored for 24 h** before gradual rollout to 25%, 50%, and 100%.
- A/B testing framework: two variants of the GLM system prompt (e.g., "price-first" vs "risk-balanced") can be evaluated by tagging recommendation_runs with a variant_id column, then comparing confidence scores and farmer feedback ratings across cohorts.

4.1.2 Emergency Rollback (ER) & Golden Release

Each Docker image build is tagged with the Git commit SHA. The last known-good image is the Golden Release. In case of anomalies:

- **P1 trigger:** GLM error rate > 5% over 10 minutes → Celery workers continue; API falls back to deterministic recommendation engine automatically (no manual intervention needed , GLMDecisionClient._fallback_recommendation)
- **P1 trigger:** PostgreSQL connection failure → FastAPI returns HTTP 503 with detail message; Render.com health checks detect unhealthy state and restart the container
- **P2 trigger:** FAMA scraper consistently returning empty results > 1 h → operator runs POST /api/v1/ops/sync {source: 'market'} to force a Firecrawl-backed refresh
- **Golden Release rollback:** docker compose down && docker compose up ,d ,, tag=<golden_sha>

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.1.3 Priority Matrix

Priority	Condition	Impact	Action
P1 Critical	GLM API (api.ilmu.ai) returns 5xx for > 5 minutes	Core recommendation engine unavailable	Automatic: GLMDecisionClient falls back to deterministic engine. Alert team. No rollback needed.
P1 Critical	PostgreSQL container down or connection pool exhausted	All persistent operations fail; 503s returned to users	Render.com auto-restart. If persistent: rollback to Golden Release. Data safe , postgres_data volume persisted.
P2 High	GDELT API rate-limited or unavailable for > 30 min	News signal missing; recommendation uses fallback message	Automatic: CompositeNewsClient switches to FirecrawlNewsClient. No manual action.
P2 High	FAMA pages return no parseable price rows	Market prices fall back to seeded historical ranges	Automatic fallback + note appended to recommendation. Operator can trigger POST /api/v1/market/refresh to retry.
P3 Medium	Open-Meteo geocoding fails for a specific location string	WeatherLocationNotFoundError thrown; weather signal skipped	Automatic: DecisionOrchestrator._fetch_weather returns None; weather_signal set to fallback message. No data loss.
P3 Medium	Celery worker fails (Redis unreachable)	Scheduled daily market refresh stops; manual refresh still works via REST	Render.com restarts Redis container. Operator re-queues via POST /api/v1/ops/sync.

4.2 Monitoring

© UMHackathon 2026

Email: umhackathon@um.edu.my

The system exposes a readiness endpoint at GET /api/v1/health/ready which checks:

- **PostgreSQL:** executes SELECT 1 and reports ok or error:{message}
- **Redis:** pings the configured Redis URL and reports ok or error:{message}
- **GLM config:** reports ok if ILMU_API_KEY is set, missing_api_key otherwise
- **Weather / news providers:** reported as configured (liveness checked at first call)

Planned production monitoring (post-hackathon):

- **Render.com** native health check on /api/v1/health with 30 s interval, 3 failure threshold → auto-restart
- **Celery Flower** dashboard (accessible via BULL_AUTH_KEY) for worker queue visibility
- **Structured logging via Python logging** module , all LLM request durations, error types, and data source failures are logged at WARNING or ERROR level
- **PostgreSQL**: pg_stat_activity query on connection count; alert if > 80% of max_connections

4.3 Assumptions

© UMHackathon 2026

Email: umhackathon@um.edu.my

The following operational and environmental conditions are assumed to hold during development and deployment:

1. Farmers have **access to a device** (smartphone or desktop) with a stable internet connection during the session.
2. **ILMU_API_KEY** is **valid** and has **sufficient credits** for the expected number of analysis runs per day.
3. The Malaysian locations entered by farmers can be resolved by Open-Meteo's geocoding API. The system validates all location strings against a known Malaysian district/state dictionary before sending to Open-Meteo.
4. FAMA's public web pages (fama.gov.my) **retain** their **current** table-based **HTML structure**. If FAMA updates their frontend, the Firecrawl fallback provides continuity.
5. GDELT's free Doc API **remains available** and **returns** results within **10 s.** Rate limiting is handled by the 10 s timeout and fallback to Firecrawl.
6. **Docker and Docker Compose are available on the deployment host**. No Kubernetes or cloud-managed database is required for the hackathon demo.
7. The **demo** farmer profile (farmer_id: demo-farmer) is **seeded at startup** and used for all hackathon demonstrations.

4.4 External Dependencies

© UMHackathon 2026

Email: umhackathon@um.edu.my

Tool / Service	Purpose	Risk Level	Mitigation
ILMU GLM 5.1 (api.ilmu.ai)	Core LLM reasoning engine. Handles field extraction from farmer free-text (intake) and evidence-grounded decision synthesis (recommendation). OpenAI-compatible API.	HIGH , If GLM API is rate-limited or down, the core product value is reduced	Deterministic fallback recommendation engine in GLMDecisionClient. Field extractor has a regex-based fallback for common patterns. System degrades gracefully rather than returning errors.
Open-Meteo API	7-day weather forecast + geocoding for all Malaysian locations. Free, no auth required.	LOW , High uptime; free tier; no API key risk	If location cannot be geocoded, WeatherLocationNotFoundError is caught and weather_snapshot = None. Weather signal falls back to a configurable message.
GDELT Doc API	Real-time agricultural news retrieval for Malaysian crops and regions. Free, no auth.	MEDIUM , Can be slow or return sparse results for niche queries	CompositeNewsClient falls back to FirecrawlNewsClient if GDELT returns empty. Final fallback is an empty news list with a neutral signal score.
FAMA Website (fama.gov.my)	Official Malaysian agricultural market prices. Scraped via HTTP + BeautifulSoup.	MEDIUM , HTML structure changes will break the scraper	FAMAMarketPriceClient → FirecrawlMarketClient → seeded fallback price ranges. All three layers are in place.
Self-hosted Firecrawl + Playwright	Secondary scraping for prices and news when primary sources fail. Runs as Docker sidecar.	MEDIUM , RabbitMQ startup and Playwright container can be slow to initialise	doctor.sh pre-flight script checks for port conflicts. Firecrawl is a fallback only , primary system works without it.
Render.com Hosting	Hosts the Docker Compose stack online for the demo URL.	MEDIUM , Free tier has cold-start latency; paid tier mitigates this	docker compose up ,,build is documented for local fallback demo. Golden Release tags allow fast re-deploy.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Redis 7	**Celery message broker + result backend for background jobs.**	**LOW , Lightweight; restarts fast**	**Celery workers retry failed tasks. API falls back to synchronous ops (market refresh via REST) if Redis is down.**
PostgreSQL 16	**All persistent data. Data volume is small for the hackathon.**	**LOW , Mature, stable; postgres_data volume survives restarts**	**postgres_data Docker volume persists data across restarts. Regular pg_dump recommended for production.**

5. Project Management & Team Contributions

5.1 Project Timeline

© UMHackathon 2026

Email: umhackathon@um.edu.my

Day	Activity	Deliverable
Day 1	Mentorship Session with Mr Izzat H. Azmi	Architecture guidance and scope confirmation
Day 2	Idea Brainstorm , defined the Pivot/Persevere/Harvest Early decision framework, chose ILMU GLM, identified FAMA + GDELT + Open-Meteo as data sources	Product framing document (GEMINI.md + plan.md)
Day 3	Project planning and role assignment (3 Frontend | 2 Backend). Database schema design. Docker Compose skeleton created.	Repo structure, docker-compose.yml, .env.example, pyproject.toml
Days 4–5	Parallel development: Backend builds intake, orchestrator, GLM client, DB repositories, Celery workers. Frontend builds index.html chat, intake flow, market page styling.	Working POST /api/v1/intake and POST /api/v1/analysis/decision locally
Day 6	Frontend completed: markets.html live data integration, mydata.html profile + recommendation display, crops.html reference guide, SSE streaming progress log	All 5 frontend pages functional with backend integration
Day 7	Backend completed: FAMA scraper + Firecrawl fallback, Celery scheduler, advisor service, feedback endpoint, ops/sync endpoints, health/readiness	Full Docker stack passing doctor.sh, /health/ready returning ok
Day 8	Integration testing, SAD documentation, demo recording, final submission	SAD PDF, demo URL on Render.com, tagged GitHub release

5.2 Team Members & Roles

© UMHackathon 2026

Email: umhackathon@um.edu.my

Name	Role	Responsibilities
Syazwan Hilman Bin Samsul	Frontend Developer (Team Leader)	Project coordination and timeline management. UI/UX oversight across all 5 pages. Integration of frontend with backend /api/v1 contract. Final deployment to Render.com.
Mohd Harith Bin Abd Manan	Frontend Developer Lead	Server deployment configuration on Render.com. HTML structure and layout design system (CSS variables, dark mode, responsive grid). Node.js proxy server (server.js) configuration.
Muhammad Aliff Bin Mohd Zaifuddin	Frontend Developer	Dashboard visualisation modules (markets.html price table, trend bars, signal cards). My Data page (mydata.html) with SSE progress log and profile display. Crops reference page (crops.html).
Mohamedelhassan Mohamed Elhag Momen	Backend Developer Lead	AI/GLM service integration (GLMDecisionClient, GLMFieldExtractor, AdvisorService). FastAPI backend architecture and orchestrator (DecisionOrchestrator). Database schema and Repository pattern. Celery worker design. Docker Compose stack.
Muhammad Aiman Zafran Bin Azlin	Backend Developer	Data flow handling and external API clients (Open-Meteo weather, GDELT news, FAMA price scraper, Firecrawl). API endpoint testing. Docker configuration and doctor.sh pre-flight tooling. Signal snapshot and sync job repositories.

5.3 Recommendations

© UMHackathon 2026

Email: umhackathon@um.edu.my

The following enhancements are recommended for the next iteration to improve scalability, reliability, and performance:

Area	Recommendation	Rationale
Pricing Data	Replace seeded fallback + FAMA scraper with a real-time normalised price feed (e.g., DOA/FAMA official data API or a dedicated price ingestion pipeline)	Current scraper is fragile to HTML changes. A reliable price feed is the single biggest accuracy gap in the recommendation pipeline.
Caching	Add Redis caching for GDELT news responses (TTL: 1 h) and Open-Meteo weather responses (TTL: 30 min) keyed by crop+region+date	Reduces external API calls during concurrent analysis runs, lowers latency, and reduces cost.
Vector Search	Add pgvector to PostgreSQL and embed stored news articles for hybrid retrieval	Enables semantic search over historical agricultural advisories, allowing the recommendation to reference past similar events.
Auth	Implement OAuth2 / JWT for multi-farmer accounts, replacing the single demo-farmer ID	Required before any real-world deployment. Enables per-farmer history and personalised recommendations.
Observability	Add OpenTelemetry traces to all GLM calls and external API fetches; ship to Grafana Cloud	Provides end-to-end latency visibility, helps identify slow data sources, and quantifies GLM token costs per run.
Frontend Build	Migrate from Vanilla JS to React + Vite for component reuse and better state management as feature count grows	The current five-page static approach works for MVP but will become unwieldy as the feature set expands.

© UMHackathon 2026

Email: umhackathon@um.edu.my